Лабораторная работа № 1

Среда разработки, язык Dart и выбор темы FitTrack

Проверки выполнены 19.10.2026. Целевая платформа - Android. Flutter 3.47.5, Dart 3.13.4.

Часть A. Среда разработки

Контрольная команда: flutter doctor

```
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.5, on macOS 26.6.2 25G83 darwin-arm64, locale ru-MD)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[!] Xcode - develop for iOS and macOS
    ✗ Xcode installation is incomplete; a full installation is necessary for iOS and macOS development.
      Download at: https://developer.apple.com/xcode/
      Or install Xcode via the App Store.
      Once installed, run:
        sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer
        sudo xcodebuild -runFirstLaunch
    ! CocoaPods not installed.
        CocoaPods is a package manager for iOS or macOS platform code.
        Without CocoaPods, plugins will not work on iOS or macOS.
        For more info, see https://flutter.dev/to/platform-plugins
      For installation instructions, see https://guides.cocoapods.org/using/getting-started.html#installation
[✗] Chrome - develop for the web (Cannot find Chrome executable at /Applications/Google
Chrome.app/Contents/MacOS/Google Chrome)
    ! Cannot find Chrome. Try setting CHROME_EXECUTABLE to a Chrome executable.
[✓] Connected device (2 available)
[✓] Network resources

! Doctor found issues in 2 categories.
```

Android toolchain и подключённое Android-устройство отмечены как готовые. Предупреждения по Xcode/CocoaPods и Chrome сохранены в полном выводе: iOS/macOS и веб в данном Android-сценарии не проверялись. Лицензии Android SDK приняты пользователем; соответствующее предупреждение устранено.

Часть A. Устройство и запуск demo

Контрольная команда: flutter devices

```
Found 2 connected devices:
  sdk gphone64 arm64 (mobile) • emulator-5554 • android-arm64 • Android 15 (API 35) (emulator)
  macOS (desktop)             • macos         • darwin-arm64  • macOS 26.6.2 25G83 darwin-arm64

Checking for wireless devices...

No wireless devices were found.

Run "flutter emulators" to list and start any available device emulators.

If you expected another device to be detected, please run "flutter doctor" to diagnose potential issues. You may
 also try increasing the time to wait for connected devices with the "--device-timeout" flag. Visit
https://flutter.dev/setup/ for troubleshooting tips.
```

Проверенное устройство: sdk gphone64 arm64, emulator-5554; AVD pixel_pam, Android 15 (API 35).

Приложение hello_pam действительно запущено на этом эмуляторе командой flutter run -d emulator-5554. Debug APK успешно собран и установлен. Три нажатия кнопки + изменили счётчик с 0 на 3. После команды hot reload (r) приложение продолжило работу со значением 3.

Ключевой фактический вывод запуска

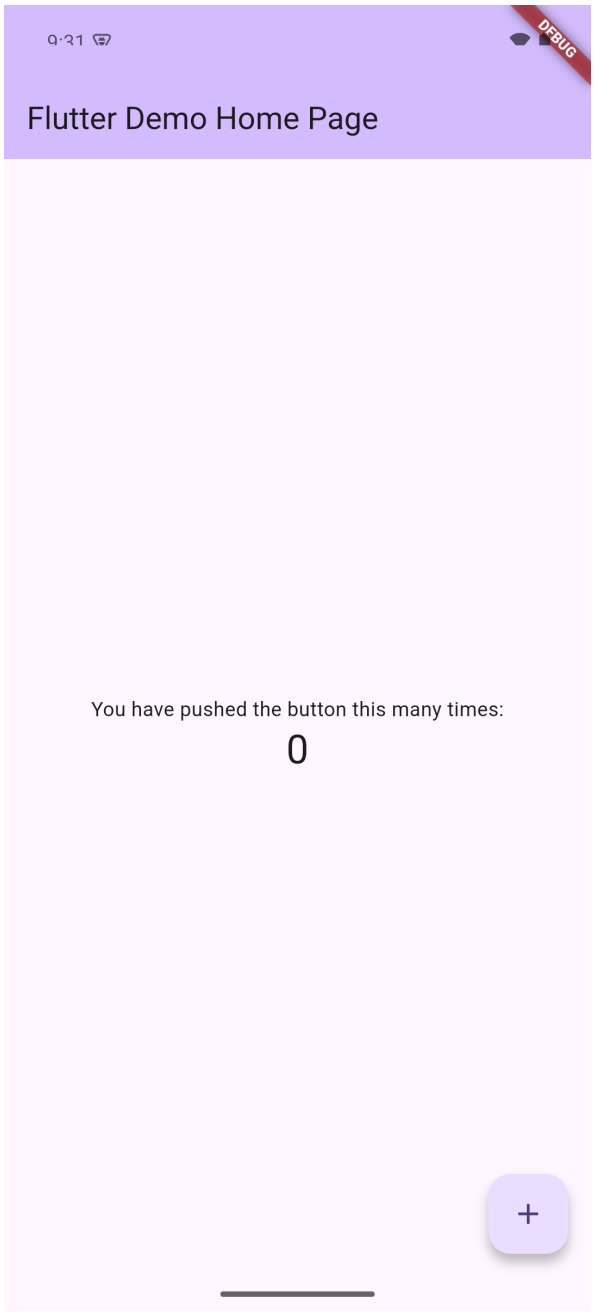
```
Built build/app/outputs/flutter-apk/app-debug.apk
Installing build/app/outputs/flutter-apk/app-debug.apk
Reloaded 0 libraries in 65ms (compile: 5 ms, reload: 0 ms, reassemble: 22 ms).
```

Во время Gradle-сборки появилось предупреждение о версиях SDK XML 3/4. Оно не остановило сборку и установку. Вывод не трактуется как ошибка приложения.

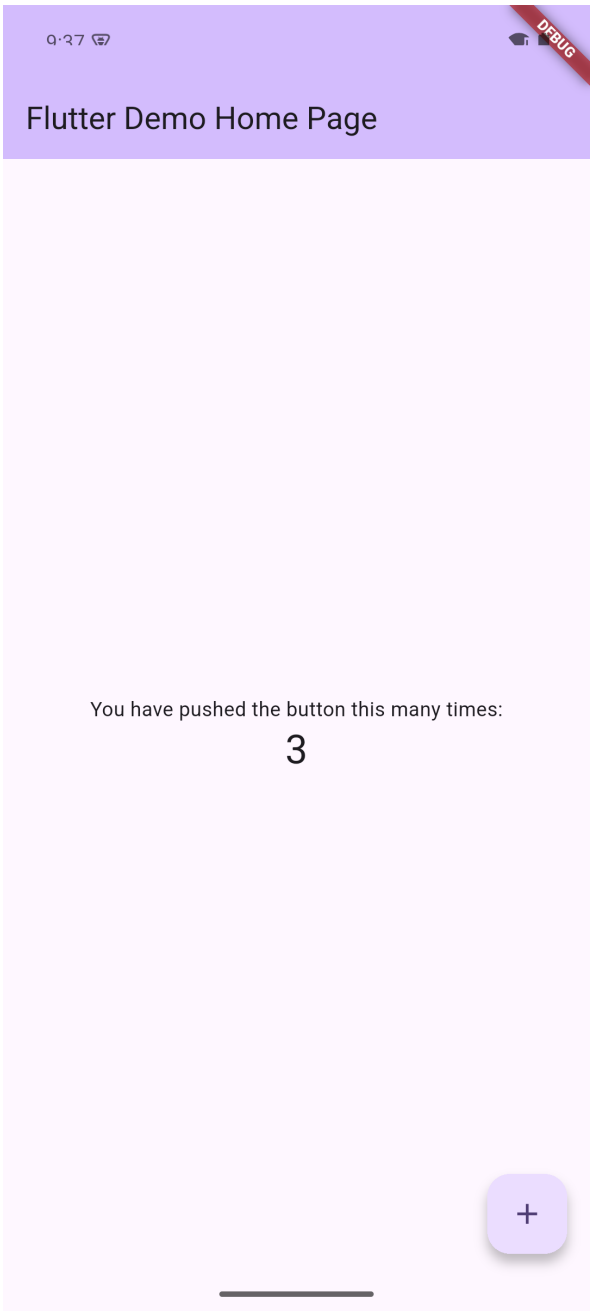
Часть A. Проверка счётчика

Снимки сделаны с работающего Android-эмулятора. Слева - начальное значение 0. Справа - значение 3 после трёх нажатий и hot reload.

До нажатий



После нажатий и hot reload



Результат: сборка, установка, запуск, обработка нажатий и сохранение состояния при hot reload подтверждены. Эти действия выполнены в demo hello_pam; FitTrack начнёт реализовываться на L2.

Часть B. E1. Переменные и null safety

Проверяется безопасный разбор целого числа: null и неверная строка возвращают fallback. Оператор ! не нужен.

Исходный код: dart_exercises/e1_null_safety.dart

```
// E1. Переменные и null safety.

int parseOrDefault(String? input, int fallback) {
  // int.tryParse принимает только String (не String?), поэтому null
  // отсекаем заранее. После этой проверки Dart сам «сужает» тип input
  // до String, и принудительное снятие nullability не нужно.
  if (input == null) return fallback;

  // tryParse не бросает исключение, а возвращает null для строк вроде
  // 'abc' или '7.5' — ?? подставляет fallback именно в этом случае.
  return int.tryParse(input) ?? fallback;
}

void main() {
  print("parseOrDefault('42', 0) → ${parseOrDefault('42', 0)}");
  print("parseOrDefault('abc', 0) → ${parseOrDefault('abc', 0)}");
  print("parseOrDefault(null, 7) → ${parseOrDefault(null, 7)}");
  print("parseOrDefault('7.5', -1) → ${parseOrDefault('7.5', -1)}");
}
```

Запуск: dart run e1_null_safety.dart

```
parseOrDefault('42', 0) → 42
parseOrDefault('abc', 0) → 0
parseOrDefault(null, 7) → 7
parseOrDefault('7.5', -1) → -1
```

Код завершения: 0. Вывод сопоставлен с примерами задания.

Часть B. E2. Коллекции и функциональные методы

Проходные оценки выбираются через where; среднее вычисляется без циклов for/while. Для пустого набора возвращается 0.0.

Исходный код: dart_exercises/e2_collections.dart

```
// E2. Коллекции и функциональные методы.

double promotedAverage(List<int> grades) {
  // where ленивый: без toList фильтр заново проходил бы по оценкам
  // при каждом обращении (isEmpty, reduce, length).
  final passed = grades.where((g) => g >= 5).toList();

  // reduce на пустом списке бросает StateError, поэтому пустой случай
  // обрабатываем до него — заодно это требование ТЗ «вернуть 0».
  if (passed.isEmpty) return 0.0;

  final sum = passed.reduce((a, b) => a + b);

  // Оператор / в Dart всегда возвращает double, даже для двух int,
  // поэтому явное приведение типа не нужно.
  return sum / passed.length;
}

void main() {
  print(
    'promotedAverage([9, 4, 7, 10, 3, 5]) → '
    '${promotedAverage([9, 4, 7, 10, 3, 5])}',
  );
  print('promotedAverage([10, 10, 9]) → ${promotedAverage([10, 10, 9])}');
  print('promotedAverage([3, 4, 2]) → ${promotedAverage([3, 4, 2])}');
}
```

Запуск: dart run e2_collections.dart

```
promotedAverage([9, 4, 7, 10, 3, 5]) → 7.75
promotedAverage([10, 10, 9]) → 9.666666666666666
promotedAverage([3, 4, 2]) → 0.0
```

Код завершения: 0. Вывод сопоставлен с примерами задания.

Часть B. E3. Именованные и необязательные параметры

Позиционный amount дополняется параметрами currency и decimals со значениями по умолчанию; округление выполняет toStringAsFixed.

Исходный код: dart_exercises/e3_functions.dart

```
// E3. Функции с именованными и необязательными параметрами.

// Параметры в {} именованные: при вызове пишется имя (currency: 'EUR'),
// поэтому их можно передавать в любом порядке или не передавать вовсе —
// тогда берутся значения по умолчанию.
String formatPrice(double amount, {String currency = 'MDL', int decimals = 2}) {
  // toStringAsFixed округляет, а не отбрасывает знаки: 99.999 → "100.00".
  return '${amount.toStringAsFixed(decimals)} $currency';
}

void main() {
  print('formatPrice(12.5) → "${formatPrice(12.5)}"');
  print(
    "formatPrice(12.5, currency: 'MDL') → "
    '"${formatPrice(12.5, currency: 'MDL')}"',
  );
  print(
    "formatPrice(99.999, currency: 'EUR') → "
    '"${formatPrice(99.999, currency: 'EUR')}"',
  );
  print(
    "formatPrice(7.5, currency: 'USD', decimals: 1) → "
    '"${formatPrice(7.5, currency: 'USD', decimals: 1)}"',
  );
}
```

Запуск: dart run e3_functions.dart

```
formatPrice(12.5) → "12.50 MDL"
formatPrice(12.5, currency: 'MDL') → "12.50 MDL"
formatPrice(99.999, currency: 'EUR') → "100.00 EUR"
formatPrice(7.5, currency: 'USD', decimals: 1) → "7.5 USD"
```

Код завершения: 0. Вывод сопоставлен с примерами задания.

Часть B. E4. Классы и равенство

Student содержит final-поля, основной и именованный конструкторы. Согласованные == и hashCode позволяют Set объединить равные значения.

Исходный код: dart_exercises/e4_classes.dart

```
// E4. Классы.

class Student {
  final String name;
  final String group;

  Student(this.name, this.group);

  // Перенаправляем в основной конструктор, чтобы логика создания
  // была в одном месте, а guest() только задавал значения.
  Student.guest() : this('Guest', '—');

  @override
  String toString() => 'Student($name, $group)';

  // По умолчанию == сравнивает ссылки (один и тот же объект в памяти),
  // а нам нужно равенство по содержимому.
  @override
  bool operator ==(Object other) =>
      other is Student && other.name == name && other.group == group;

  // Set и Map сначала сравнивают hashCode, и только при совпадении
  // вызывают ==. Равные объекты обязаны иметь одинаковый hashCode,
  // поэтому считаем его из тех же полей, что участвуют в ==.
  @override
  int get hashCode => Object.hash(name, group);
}

void main() {
  print(Student('Ana Rusu', 'TI-231'));
  print(Student.guest());
  print(
    "Student('Ana Rusu', 'TI-231') == Student('Ana Rusu', 'TI-231') → "
    '${Student('Ana Rusu', 'TI-231') == Student('Ana Rusu', 'TI-231')}',
  );
  print(
    "{Student('Ana Rusu', 'TI-231'), Student('Ana Rusu', 'TI-231')}.length → "
    '${{Student('Ana Rusu', 'TI-231'), Student('Ana Rusu', 'TI-231')}.length}',
  );
}
```

Запуск: dart run e4_classes.dart

```
Student(Ana Rusu, TI-231)
Student(Guest, —)
Student('Ana Rusu', 'TI-231') == Student('Ana Rusu', 'TI-231') → true
{Student('Ana Rusu', 'TI-231'), Student('Ana Rusu', 'TI-231')}.length → 1
```

Код завершения: 0. Вывод сопоставлен с примерами задания.

Часть B. E5. Mixin Loggable

Один mixin переиспользуется двумя сервисами. runtimeType формирует имя класса в префиксе сообщения.

Исходный код: dart_exercises/e5_mixin.dart

```
// E5. Mixin.

// Mixin, а не базовый класс: у класса в Dart может быть только один
// родитель, а mixin-ов можно подключить сколько угодно. Так сервисы
// получают логирование, не тратя на него своё наследование.
mixin Loggable {
  // runtimeType — реальный класс объекта во время выполнения, поэтому
  // один и тот же код печатает CartService или AuthService.
  void log(String msg) => print('[$runtimeType] $msg');
}

class CartService with Loggable {
  void addItem(String item) => log('товар добавлен: $item');
}

class AuthService with Loggable {
  void login(String username) => log('пользователь вошёл: $username');
}

void main() {
  CartService().addItem('Paracetamol');
  AuthService().login('ana.rusu');
}
```

Запуск: dart run e5_mixin.dart

```
[CartService] товар добавлен: Paracetamol
[AuthService] пользователь вошёл: ana.rusu
```

Код завершения: 0. Вывод сопоставлен с примерами задания.

Часть B. E6. Асинхронное программирование

Future.delayed имитирует двухсекундное ожидание. await получает строку результата до печати сообщения Готово.; сам event loop не блокируется.

Исходный код: dart_exercises/e6_async.dart

```
// E6. Асинхронное программирование.

Future<String> fetchProfile(String username) async {
  // Future.delayed не блокирует программу, а только планирует
  // продолжение через 2 секунды — так имитируется ответ сервера.
  await Future.delayed(const Duration(seconds: 2));
  return 'Профиль: $username, группа TI-231, решено упражнений: 42';
}

// main объявлен async, иначе внутри него нельзя использовать await.
Future<void> main() async {
  print('Загружается профиль пользователя ana.rusu...');

  // await приостанавливает main до готовности Future, поэтому
  // «Готово.» гарантированно печатается после профиля.
  final profile = await fetchProfile('ana.rusu');
  print(profile);

  print('Готово.');
}
```

Запуск: dart run e6_async.dart

```
Загружается профиль пользователя ana.rusu...
Профиль: ana.rusu, группа TI-231, решено упражнений: 42
Готово.
```

Код завершения: 0. Вывод сопоставлен с примерами задания.

Часть B. Итог проверки

Проверка	Результат
Формат без изменения файлов	Formatted 6 files (0 changed) in 0.01 seconds.; exit 0
Статический анализ	Analyzing .... No issues found!; exit 0
Запуски E1-E6	6 из 6, exit 0; вывод совпал с заданием
Исходники упражнений	Не изменялись при подготовке отчёта

Уточнения для объяснения кода

Type promotion применяется к параметрам и локальным переменным; начиная с Dart 3.2 - также к подходящим private final-полям. Публичные и изменяемые поля могут не сужаться.

Если удалить await только перед fetchProfile в E6, переменная получит Future<String>. Напечатается представление Future и сообщение Готово.; завершившийся позже результат сам не выводится, поскольку внутри fetchProfile нет print. Это поведение проверено на временной копии.

const помогает переиспользовать экземпляры конфигурации Flutter-виджетов. Он не запрещает все перестроения при изменении State или inherited-данных.

Приложение к отчёту содержит мини-спецификацию T2 FitTrack на одной странице. Устная защита и понимание кода подтверждаются студентом отдельно.

Приложение. Мини-спецификация FitTrack

Тема

T2. FitTrack — дневник тренировок. Домен: здоровье и фитнес. Приложение будет строиться на L2–L6; сейчас это план, кода приложения ещё нет.

Целевая аудитория и проблема

Студенты, которые регулярно тренируются и хотят видеть свою динамику. Бумажную тетрадку легко забыть или потерять в раздевалке, и история тренировок пропадает. FitTrack хранит каталог упражнений и дневник сессий в телефоне, поэтому прогресс всегда под рукой.

Экраны

1. **Вход / регистрация** — один экран с двумя режимами.
2. **Каталог упражнений** — список с поиском и фильтром по группе мышц.
3. **Карточка упражнения** — описание, техника выполнения, изображение.
4. **Дневник тренировок** — список сессий в хронологическом порядке.
5. **Форма сессии** — создание и редактирование: дата, упражнения, длительность, заметки.
6. **Статистика прогресса** — сессии в неделю, суммарная длительность.
7. **Профиль пользователя.**

Сущности

Поля совпадают с моделью данных темы. Типы предварительные, их сверю с backend на L5.

Exercise — упражнение из каталога.

Поле	Тип	Значение
`id`	`String`	идентификатор
`name`	`String`	название
`muscleGroup`	`String`	группа мышц, по ней работает фильтр
`description`	`String`	описание и техника
`imageUrl`	`String`	ссылка на изображение

WorkoutSession — одна тренировка в дневнике.

Поле	Тип	Значение
`id`	`String`	идентификатор
`userId`	`String`	владелец сессии
`date`	`DateTime`	дата тренировки
`exerciseIds`	`List<String>`	выполненные упражнения (ссылки на `Exercise.id`)
`durationMin`	`int`	длительность в минутах
`notes`	`String`	заметки

Почему эта тема

Я сам спортсмен, поэтому для меня это приложение будет полезным не только как учебный проект. Мне важно видеть, сколько и как регулярно я тренируюсь, а дневник в телефоне удобнее записей, которые легко потерять. Делая FitTrack, я смогу сразу проверять его на своих тренировках и замечать, чего в нём не хватает.